

Evolution status after San Diego 2018

Abstract

This paper is a collection of items Evolution has worked on up to and in the San Diego meeting, their status, and plans for the future.

Concepts

We reviewed [P1141R1](#), Yet another approach for constrained declarations, and approved it. Core moved it in San Diego. In addition, we approved [P0848R0](#), Conditionally Trivial Special Member Functions.

We did not look at [P0782R2](#), Constraining Concepts Overload Sets, despite it being on the agenda. This proposal changes dependent calls in constrained templates to consider only the overloads that were used to satisfy a constraint as viable candidates. We will possibly revisit the proposal in Kona, as we need to decide the fate of this change before C++20 ships.

Modules

The merged Modules proposal, [P1103R1](#) was approved, with a bunch of fixes, including

- Making the module and import keywords context-sensitive
- Support for inline module partitions
- Change to when a module preamble is considered to end
- Fixes to redefinitions in legacy imports

The global module fragment was not removed, and the paper on retiring pernicious language constructs in module contexts was rejected.

The plan has been for quite some time to wait until Kona before moving Modules, so as to give implementers more time to gain implementation experience.

Coroutines

We discussed [P1063R1](#), Core Coroutines. Evolution, again, still, has consensus (but far from unanimity) to move the merge of the Coroutines TS forward.

Aggregate initialization

The proposal to allow paren-initializing aggregates, [P0960](#), was approved.

Contracts

The change to allow preconditions and postconditions to use private members, [P1289R0](#), was approved.

We discussed UB and optimizations at quite some length; the work on that problem is ongoing, and we'll see more in Kona.

Some constexpr extensions

- [P1327](#), allowing `dynamic_cast` in constant expression was approved. It's also possible to use `typeid` and its comparison in constant expressions.
- We approved being able to change the active member of a union in constant expressions.
- We approved [P1073](#), "constexpr".
- We discussed "for..." and "for constexpr". Further work is expected with encouragement that we want facilities like that.

Template argument deduction

We approved fixes that allow CTAD on aggregates, alias templates, and using inherited constructors.

All Your Spaceships Are Belong to Us

The papers [P1185](#) and [P1186](#) were approved. We expect further discussion on spaceship in Kona.

There was no consensus on making comparison operators chain, nor was there encouragement for further work in that area.

Structured bindings

We discussed removing the need for `tuple_element` and allowing explicit typing of bindings. There was no consensus for either; Pattern Matching can solve the latter problem.

Defaulted special member functions and noexcept

We approved [P1286](#), which changes `noexcept` on defaulted special member functions to no longer require the `noexcept-spec` to match what would've been implicitly generated.

Guaranteed constant initialization for non-constexpr variables

We discussed [P1143](#), Adding the `[[constinit]]` attribute, and we expect to see a revision in Kona. The current guidance is for a keyword instead of an attribute.

Deprecations

Deprecating `volatile` was discussed; a revised proposal is expected for Kona.

[P1161](#), Deprecate uses of the comma operator in subscripting expressions, was approved. This paves the way to multi-argument subscripting. We didn't approve making that possible yet, but the deprecation is approved.